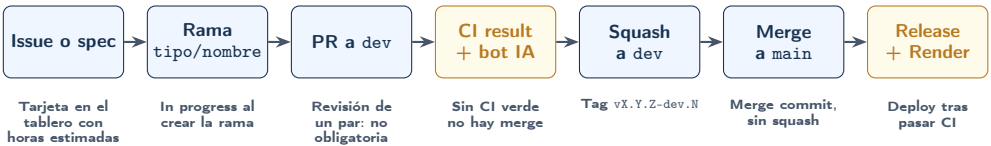
Ciclo de desarrollo: de la tarjeta a producción



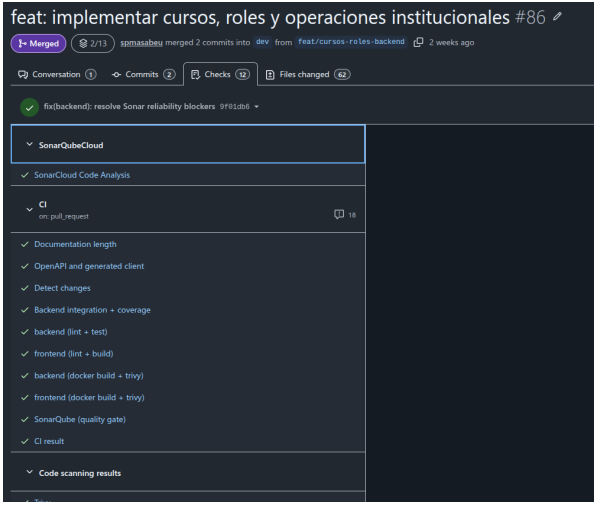
68
PR integradas desde el 11/10

54 / 13
a dev / a main

23 de 68
con revisión de un par

Reglas en GitHub: `dev` solo acepta squash; `main` solo merge commit. Ambas exigen el check `CI result` y bloquean force push y borrado.

Una PR real: #86, cursos y roles



- Título en conventional commits: define la versión del release.
- De `feat/cursos-roles-backend` a `dev`.
- Enlaza el spec y la evidencia de validación, y cierra 4 issues del backlog.
- 12 checks en verde: CI, SonarQube y Trivy.
- `CI result` habilita el merge.

Prácticas que usamos y qué nos dieron

Práctica	Evidencia	Beneficio observado
Spec Kit antes de codificar	`specs/` por funcionalidad, con CU y RF	El alcance de cada PR está escrito antes del código
ADR	22 decisiones en `docs/adr/`	El cambio de Railway a Render quedó trazado con contexto y costos
Conventional commits	67 tags y releases automáticos	Cada versión en Render se identifica en `/health`
CI como gate	`CI result` obligatorio en `dev` y `main`	35 ejecuciones fallidas en PR detuvieron el merge hasta corregir
Tests de aislamiento	UC y UTFSM, RLS y E2E de tenant	El aislamiento se prueba en cada PR, no solo antes de la demo
Tablero con horas	Backlog GitHub Projects	Proyección del pendiente en horas

Pendiente de mejora: la revisión de un par no es obligatoria (34 % de las PR la tuvieron) y la etiqueta `bug` no se usa.